# PFPF: Python vs MATLAB Implementation Differences

**Date**: 2026-02-10 **Last Updated**: 2026-02-10 (Revised after code-level verification)

This document compares the Python/TensorFlow implementation (`code/`) against the published MATLAB implementation (`PFPF/`). Each issue was verified by reading both codebases line-by-line.

## Priority Legend

- **HIGH** - Genuine algorithmic difference from MATLAB; likely impacts results
- **MEDIUM** - Real difference but impact depends on problem conditioning
- **LOW** - Minor efficiency or code-cleanliness issue

## MATLAB Codebase Structure

The MATLAB `PFPF/` folder contains **two separate implementations**:

1. **Standalone non-invertible filter**: `DH_ExactFlow_Filter.m` – runs the entire EDH filter (propagation, flow, update) in one file with equal weights.
2. **Modular invertible PFPF**: `PFPF.m` → `propagateAndEstimatePriorCovariance.m` → `particleFlow.m` → `correctoinAndCalculateWeights.m` – the full invertible algorithm with Jacobian tracking and importance weights.

The Python filters correspond to the **modular PFPF** variant. All MATLAB references below point to the correct corresponding file.

---

# Part A: Shared Issues (Utility Modules)

These affect multiple or all particle flow filters.

---

## Issue #1: HIGH - Regularization Applied to S Instead of P

### Affected Filters

All filters using `compute_flow_params`.

### Problem

Python regularizes $S = \lambda HPH^T + R$ directly (`flow_params.py:62-67`), while MATLAB regularizes $P$ (the covariance) before the flow computation and never touches $S$.

### MATLAB Reference

MATLAB regularizes $P$ conditionally – only when Cholesky fails:

```
% propagateAndEstimatePriorCovariance.m, lines 52-55 (EDH)
[~,regind] = chol(vg.PP);
if regind
    vg.PP = cov_regularize(vg.PP);
end

% propagateAndEstimatePriorCovariance.m, lines 65-68 (LEDH, per-particle)
[~,regind] = chol(squeeze(vg.PP_all(:,:,particle_ix)));
if regind
```

```
    vg.PP_all(:,:,particle_ix) = cov_regularize(squeeze(vg.PP_all(:,:,particle_ix)));
end
```

During the flow itself, MATLAB uses $P$ directly with no regularization on $S$:

```
% homotopy_Mean.m, line 41 (EDH)
A = -0.5*S*((lambda*H*S+ps.likeparams.R)\H);

% homotopy_Local.m, line 101 (LEDH)
A_i = -0.5*PP_HiTranspose*((lambda*Hi*PP_HiTranspose+Ri)\Hi);
```

### Current Python Code

**File**: `code/src/utils/flow_params.py`, lines 62-67

```
# Regularization applied to S (not P)
if regularization > 0.0:
    min_diag = tf.reduce_min(tf.linalg.diag_part(S))
    min_diag = tf.maximum(min_diag, regularization)
    reg_strength = tf.maximum(min_diag * 0.01, regularization)
    S = S + reg_strength * tf.eye(tf.shape(S)[0], dtype=tf.float32)
```

### Why This Matters

Regularizing $S$ vs $P$ are not equivalent: - Regularizing $P$: $S = \lambda H(P + \varepsilon I)H^T + R$ – increases prior uncertainty - Regularizing $S$: $S = \lambda HPH^T + R + \varepsilon I$ – modifies the innovation covariance

The first preserves the mathematical relationship between $A$, $b$, and $P$; the second does not.

### Recommended Fix

Match MATLAB: regularize $P$ before passing it to `compute_flow_params`. Option A – do it inside the function:

```
P_reg = P
if regularization > 0.0:
    try:
        tf.linalg.cholesky(P)
    except:
        P_reg = P + regularization * tf.eye(state_dim, dtype=P.dtype)
HPH = H @ P_reg @ tf.transpose(H)
S = lambda_val * HPH + R
```

Option B – do it in each filter's update loop before calling `compute_flow_params`, and pass `regularization=0.0`.

**Status**: [ ] Not Started

---

## Issue #2: MEDIUM - Cholesky Regularization Strategy Differs

### Affected Filters

All filters using `safe_cholesky` in `linalg.py`.

### Problem

Python unconditionally adds a fixed jitter of `1e-6` to every matrix before Cholesky. MATLAB only regularizes when Cholesky actually fails, using a much smaller starting jitter of `1e-14` applied iteratively.

### MATLAB Reference

```
% cov_regularize.m (actual code)
function cova = cov_regularize(cova)
    dim = size(cova,1);
    reg = eye(dim,dim)*1e-14;           % Fixed 1e-14 per iteration
    [~,indicator] = chol(cova);
    count = 0; maxCount = 100;
    while indicator > 0 && count < maxCount
        cova = cova + reg;               % Iteratively add 1e-14*I
        [~,indicator] = chol(cova);
        count = count + 1;
    end
end
```

### Current Python Code

**File**: `code/src/utils/linalg.py`, lines 7-22

```
@tf.function
def safe_cholesky(A: tf.Tensor, jitter: float = 1e-6) -> tf.Tensor:
    n = tf.shape(A)[-1]
    eye = tf.eye(n, dtype=A.dtype)
    A_reg = A + eye * jitter       # Always adds 1e-6, no retry
    return tf.linalg.cholesky(A_reg)
```

### Key Differences

| Aspect | Python | MATLAB |
|---|---|---|
| When regularized | Always (unconditionally) | Only when Cholesky fails |
| Starting jitter | `1e-6` | `1e-14` |
| Retry logic | None | Up to 100 iterations |
| Total possible jitter | `1e-6` | Up to `100 * 1e-14 = 1e-12` |

Python's `1e-6` is 8 orders of magnitude larger than MATLAB's starting `1e-14`, which could mask ill-conditioning rather than minimally correcting it.

### Recommended Fix

Add conditional regularization with retry:

```
def safe_cholesky(A: tf.Tensor, jitter: float = 1e-14, max_attempts: int = 100):
    n = tf.shape(A)[-1]
    eye = tf.eye(n, dtype=A.dtype)
    reg = eye * jitter
    A_reg = A
    for attempt in range(max_attempts):
        try:
            return tf.linalg.cholesky(A_reg)
```

```
        except tf.errors.InvalidArgumentError:
            A_reg = A_reg + reg
    return tf.linalg.cholesky(A_reg)  # Last attempt
```

Note: TensorFlow's `@tf.function` tracing does not support Python `try/except` on TF ops. An eager-mode wrapper or `tf.debugging.check_numerics` approach may be needed.

**Status**: [ ] Not Started

---

## Issue #3: HIGH - Weight Clipping Not Present in MATLAB

### Affected Filters

EDH Invertible, LEDH Invertible (filters using `compute_flow_weights`).

### Problem

Python clips log weights to `[-30, 30]` after max-normalization. MATLAB does not clip at all.

### MATLAB Reference

Every weight computation in MATLAB uses the same pattern – max-subtract, exponentiate, normalize:

```
% particle_estimate.m, lines 11-14
log_weights = log_weights - max(log_weights);
ml_weights = exp(log_weights(:));
ml_weights = ml_weights / sum(ml_weights);

% correctoinAndCalculateWeights.m, lines 22-23
vg.logW = log_prior + llh - log_proposal + vg.logW;
vg.logW = vg.logW - max(vg.logW);
```

No clipping anywhere. A search for "clip" and "clamp" across the entire PFPF folder returns zero matches.

### Current Python Code

**File**: `code/src/utils/distributions.py`, line 110

```
def compute_flow_weights(
    ...,
    clip_range: Tuple[float, float] = (-30.0, 30.0)  # Default clipping
) -> tf.Tensor:
```

Called with `clip_range=(-30, 30)` in `ledh_invertible.py:293`.

### Why This Matters

After max-normalization, the largest log weight is 0 and all others are negative. Clipping at -30 means any particle whose log weight is more than 30 below the maximum gets artificially boosted by a factor of $e^{\Delta}$ where $\Delta$ could be very large. This distorts the weight distribution and can prevent the filter from properly downweighting unlikely particles.

### Recommended Fix

Remove clipping to match MATLAB:

```
# In compute_flow_weights or at call sites:
weights = normalize_log_weights(log_weights, clip_range=None)
```

**Status**: [ ] Not Started

---

## Issue #4: LOW - Redundant Step Size Normalization

### Affected Filters

All filters with `_generate_lambda_steps`.

### Problem

Python divides the step sizes by their sum after generating them. MATLAB does not, because the geometric series formula already guarantees the sum equals 1.

### MATLAB Reference

```
% generateExponentialLambda.m
lambda_1 = (1-delta_lambda_ratio) / (1-delta_lambda_ratio^nLambda);
lambda_intervals = lambda_1 * delta_lambda_ratio.^[0:nLambda-1];
lambda_range = cumsum(lambda_intervals);  % No division by sum
```

### Current Python Code

**File**: `code/src/filters/particle/ledh_invertible.py`, lines 130-135

```
def _generate_lambda_steps(self):
    q = 1.2
    epsilon_1 = (1 - q) / (1 - q**self.n_lambda_steps)
    lambda_steps_np = epsilon_1 * q**np.arange(self.n_lambda_steps)
    self.lambda_steps = lambda_steps_np / np.sum(lambda_steps_np)  # Redundant
```

Same pattern in `edh_flow.py:168-178` and other filters.

### Impact

Negligible. The geometric series formula guarantees $\sum \epsilon_j = 1$ analytically. Dividing by the sum introduces floating-point error of order $\sim 10^{-16}$, which is harmless. This is a code cleanliness issue only.

### Recommended Fix

Remove the division:

```
self.lambda_steps = epsilon_1 * q**np.arange(self.n_lambda_steps)
# Optionally verify: assert abs(np.sum(self.lambda_steps) - 1.0) < 1e-10
```

**Status**: [ ] Not Started

---

## Issue #5: MEDIUM - Missing Per-Step Jacobian Normalization (LEDH Invertible)

### Affected Filters

LEDH Invertible (and any filter tracking Jacobian determinants).

### Problem

MATLAB max-normalizes the cumulative log-Jacobian sum at **every lambda step** to prevent overflow. Python accumulates the Jacobian product without this normalization.

### MATLAB Reference

```
% particleFlow.m, lines 27-28
log_jacobian_det_sum = log_jacobian_det_sum + log_jacobian_det;
log_jacobian_det_sum = log_jacobian_det_sum - max(log_jacobian_det_sum);
```

### Current Python Code

**File**: `code/src/filters/particle/ledh_invertible.py`, lines 277-280

```
M_i = tf.eye(self.state_dim, dtype=tf.float32) + d_lambda * A_i
det_M_i = tf.linalg.det(M_i)
theta[i].assign(theta[i] * tf.abs(det_M_i))
```

The product `theta[i]` grows (or shrinks) without any normalization across steps.

### Why This Matters

For long flows (many lambda steps) or high-dimensional problems, the accumulated Jacobian product can overflow or underflow. MATLAB prevents this by subtracting the max in log-space at each step.

### Recommended Fix

Work in log-space like MATLAB:

```
# Initialize
log_theta = tf.Variable(tf.zeros(self.n_particles, dtype=tf.float32))

# Inside the flow loop, after computing A_i:
M_i = tf.eye(self.state_dim, dtype=tf.float32) + d_lambda * A_i
log_det_M_i = tf.math.log(tf.abs(tf.linalg.det(M_i)))
log_theta[i].assign(log_theta[i] + log_det_M_i)

# Max-normalize after each lambda step (outside particle loop):
max_log_theta = tf.reduce_max(log_theta)
log_theta.assign(log_theta - max_log_theta)

# Convert back when computing weights:
theta = tf.exp(log_theta)
```

**Status**: [ ] Not Started

---

### Issue #6: LOW - R_inv Recomputed Every Lambda Step (LEDH Invertible)

#### Affected Filters

LEDH Invertible.

#### Problem

`R_inv = tf.linalg.inv(R)` is computed at every lambda step. Since $R$ is constant, this should be computed once.

#### Current Python Code

**File**: `code/src/filters/particle/ledh_invertible.py`, lines 248-250

```
for j in range(self.n_lambda_steps):          # Lambda loop
    d_lambda = self.lambda_steps[j]
    lambda_val += d_lambda
    R_inv = tf.linalg.inv(R)                  # Recomputed 29 times
    regularization_tf = tf.constant(...)
    for i in range(self.n_particles):          # Particle loop
        ...
```

Note: `R_inv` is inside the lambda loop but outside the particle loop, so it is computed 29 times (once per step), not 29 x N_particles times.

#### Recommended Fix

Move before the lambda loop:

```
R_inv = tf.linalg.inv(R)                # Compute once
regularization_tf = tf.constant(self.regularization, dtype=tf.float32)
for j in range(self.n_lambda_steps):
    ...
```

The EDH Invertible filter already does this correctly (`edh_invertible.py:244-247`).

**Status**: [ ] Not Started

---

## Part B: Items Verified as Correct (No Fix Needed)

The following aspects of the Python code were verified to **match** the MATLAB implementation.

---

### Verified #1: eta_bar_0 / mu_0 Computation (ALL Filters)

#### Concern (from original report)

The original report claimed that using `global_filter.mean` after EKF predict differs from MATLAB's deterministic propagation `propagatefcn(M, propparams_no_noise)`.

#### Verification

The Python EKF predict step (`extended_kalman.py:119`) uses:

```
mean_pred = self.model.state_transition_mean(mean)
```

This is the **same** deterministic function $f(x)$ that MATLAB calls with `propparams_no_noise`. The EKF predicted mean IS $f(\bar{x})$; process noise $Q$ only affects the covariance, not the mean. Therefore:

- Python: `self.eta_bar_0 = self.global_filter.mean` after predict = $f$(ensemble_mean)
- MATLAB: `vg.mu_0 = propagatefcn(vg.M, propparams\_no\_noise)` = $f(M)$

These are identical for all filters (EDH Invertible, EDH Flow, LEDH Flow).

For LEDH Invertible (per-particle): - Python: `self.eta_bar_0[i] = model.state_transition_mean(particles_prev[i]` = $f(x^i_{k-1})$ - MATLAB: `vg.mu_0_all(:,i) = propagatefcn(vg.xp(:,i), propparams\_no\_noise)` = $f(x^i_{k-1})$

Also identical.

---

## Verified #2: Auxiliary Variable Propagation (LEDH Invertible)

### Concern (from original report)

The original report claimed that propagating both `eta_0` and `eta_bar_0` from `particles_prev` was incorrect.

### Verification

MATLAB does **exactly the same thing** – both stochastic and deterministic propagation start from `vg.xp` (the current particles before propagation):

```
% propagateAndEstimatePriorCovariance.m, lines 75-83
vg.xp_prop_deterministic = propagatefcn(vg.xp, propparams_no_noise);  % Deterministic
vg.xp_prop = propagatefcn(vg.xp, ps.propparams);                      % Stochastic
vg.mu_0_all = propagatefcn(vg.xp, propparams_no_noise);               % Same as deterministic
vg.xp_auxiliary_individual = vg.xp_prop_deterministic;
```

Python's `particles_prev` = `self.particles` at the start of predict = `vg.xp`. Both codes propagate from the same source. This is correct by design – the invertibility mechanism relies on the stochastic/deterministic split from the same starting point, with the Jacobian determinant accounting for the flow transformation.

---

## Verified #3: Covariance Symmetrization (TF-Based EKF Filters)

### Concern (from original report)

The original report claimed covariances are not symmetrized after EKF updates.

### Verification

The Python EKF (`extended_kalman.py`) already calls `symmetrize()` in **both** predict and update:

```
# _predict_step, line 125:
cov_pred = symmetrize(cov_pred)

# _update_step, line 177:
cov_updated = symmetrize(cov_updated)
```

This applies to **EDH Invertible** and **LEDH Invertible**, which use this TF-based EKF as their global/per-particle filter.

**Note**: The non-invertible flow filters (`edh_flow.py`, `ledh_flow.py`) may use a numpy-based EKF. If that EKF does not symmetrize internally, adding explicit symmetrization after `global_filter.update(y)` would be prudent. This should be verified on a case-by-case basis.

---

### Verified #4: Flow Parameter Formulas Match

The $A(\lambda)$ and $b(\lambda)$ computations were verified to match between:

- Python `compute_flow_params` in `flow_params.py`
- MATLAB `homotopy_Mean.m` (EDH) and `homotopy_Local.m` (LEDH)

Both implement:

$A = -\frac{1}{2}PH^T(\lambda HPH^T + R)^{-1}H$

$b = (I + 2\lambda A)\left[(I + \lambda A)PH^TR^{-1}(z - e) + A\mu_0\right]$

The linearization point, observation Jacobian $H$, error term $e = h(x) - Hx$, and the update of auxiliary particles during the flow all match correctly.

---

## Part C: Optional Enhancements

---

### Enhancement #1: LOW - Redraw Feature (EDH Flow)

MATLAB's standalone `DH_ExactFlow_Filter.m` implements an optional "redraw" feature that resamples particles from $\mathcal{N}(\bar{x}, P_U)$ at each timestep to maintain diversity:

```
% DH_ExactFlow_Filter.m, lines 24-25
if tt ~= 1 && ps.setup.Redraw
    vg.xp = bsxfun(@plus, sqrtm(vg.PU)*randn(dimState, nParticle), vg.xp_m);
end
```

This is not present in the Python implementation. It could be added as an optional flag but is not required for correctness.

---

## Implementation Priority

### Phase 1: Algorithmic Differences (Fix First)

| # | Issue | Priority | Files |
|---|---|---|---|
| 1 | Regularize P not S | HIGH | `flow_params.py` |
| 2 | Remove weight clipping | HIGH | `distributions.py`, `ledh_invertible.py` |
| 3 | Add Jacobian log-space normalization | MEDIUM | `ledh_invertible.py` |

### Phase 2: Numerical Robustness

| # | Issue | Priority | Files |
|---|---|---|---|
| 4 | Iterative Cholesky regularization | MEDIUM | `linalg.py` |
| 5 | Cache R_inv outside loop | LOW | `ledh_invertible.py` |
| 6 | Remove redundant step normalization | LOW | All flow filters |

### Phase 3: Optional

| # | Issue | Priority | Files |
|---|---|---|---|
| 7 | Add redraw feature | LOW | `edh_flow.py` |

---

## Verification Checklist

After implementing fixes, verify:

```python
# 1. Step sizes sum to 1
step_sum = np.sum(filter.lambda_steps)
assert abs(step_sum - 1.0) < 1e-10

# 2. No NaN or Inf in particles and weights
assert np.all(np.isfinite(filter.particles.numpy()))
if hasattr(filter, 'weights'):
    assert np.all(np.isfinite(filter.weights.numpy()))

# 3. Covariances are symmetric and positive definite
if hasattr(filter, 'particle_covs'):
    for i in range(filter.n_particles):
        P = filter.particle_covs[i]
        assert np.allclose(P, P.T, atol=1e-8)
        assert np.all(np.linalg.eigvalsh(P) > 0)

# 4. ESS does not collapse (invertible filters)
if hasattr(filter, 'ess_history'):
    ess_ratio = np.array(filter.ess_history) / filter.n_particles
    assert np.mean(ess_ratio) > 0.1, f"Mean ESS ratio: {np.mean(ess_ratio):.3f}"

# 5. Regularization targets P, not S (after fix)
# Manually verify by checking that flow_params.py no longer modifies S
```

---

## MATLAB Files Referenced

| File | Role |
|---|---|
| `PFPF/particle_flow/PFPF.m` | Main PFPF loop (invertible variant) |
| `PFPF/particle_flow/propagateAndEstimatePriorCovariance.m` | Particle propagation + EKF covariance |
| `PFPF/particle_flow/particleFlow.m` | Flow loop with Jacobian tracking |
| `PFPF/particle_flow/calculateSlope.m` | Dispatches to homotopy_Local or homotopy_Mean |
| `PFPF/particle_flow/homotopy_Local.m` | LEDH: per-particle $A(\lambda)$, $b(\lambda)$ |

| File | Role |
|---|---|
| `PFPF/particle_flow/homotopy_Mean.m` | EDH: shared $A(\lambda)$, $b(\lambda)$ |
| `PFPF/particle_flow/correctoinAndCalculateWeight.m` | Weight update + EKF update + resampling |
| `PFPF/particle_flow/DH_ExactFlow_Filter.m` | Standalone non-invertible EDH (separate algorithm) |
| `PFPF/tools/cov_regularize.m` | Iterative Cholesky regularization |
| `PFPF/tools/log_proposal_density.m` | Proposal density with Jacobian |
| `PFPF/initialization/generateExponentialLambda.m` | Exponential lambda schedule |
| `PFPF/particle_filter/particle_estimate.m` | Weight normalization and state estimation |

## Python Files to Fix

| File | Issues |
|---|---|
| `code/src/utils/flow_params.py` | #1: Regularization target |
| `code/src/utils/distributions.py` | #3: Weight clipping |
| `code/src/utils/linalg.py` | #4: Cholesky strategy |
| `code/src/filters/particle/ledh_invertible.py` | #5: Jacobian normalization, #6: R_inv caching |
| `code/src/filters/particle/edh_flow.py` | #6: Step normalization |

## Key Paper

Li, Y. and Coates, M., "Particle filtering with invertible particle flow", IEEE Transactions on Signal Processing, 2017.

---

**Document Status**: Revised after line-by-line verification against MATLAB source **Last Updated**: 2026-02-10